UNIWERSYTET WSB MERITO W WARSZAWIE

Imię i nazwisko: Vasyl Tryndiuk

Numer albumu: 159856

Tytuł pracy dyplomowej

Projekt i implementacja warstwy front-endowej aplikacji webowej do rezerwacji seansów kinowych z wykorzystaniem biblioteki React

Praca inżynierska

Promotor:
dr inż. Piotr Bobiński

Kierunek: Informatyka
Specjalność: Front-end Developer

Warszawa, 2026

SPIS TREŚCI

WSTĘP ... 3
ROZDZIAŁ 1. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ ORAZ PODSTAWY TEORETYCZNE ... 7
1.1. Systemy rezerwacyjne – analiza i funkcjonalność ... 7
1.2. Architektura aplikacji webowych i model klient–serwer ... 9
1.3. Komunikacja REST i wymiana danych w aplikacjach webowych ... 11
1.4. Technologie front-end wykorzystane w projekcie – React i zarządzanie stanem ... 13
1.5. Technologie back-end oraz baza danych w projekcie ... 15
ROZDZIAŁ 2. PROJEKT SYSTEMU INFORMATYCZNEGO ... 17
2.1. Analiza wymagań i założeń projektowych ... 17
2.2. Projekt bazy danych ... 19
2.3. Projekt architektury systemu ... 21
2.4. Diagramy UML ... 23
2.5. Projekt interfejsu użytkownika ... 27
ROZDZIAŁ 3. IMPLEMENTACJA APLIKACJI ... 28
3.1. Struktura aplikacji front-end ... 28
3.2. Logika działania aplikacji – wybór miejsc i interakcja użytkownika ... 34
3.3. Komunikacja z backendem i przetwarzanie danych ... 38
3.4. Autoryzacja i obsługa użytkownika ... 42
3.5. Analiza techniczna i możliwości rozwoju systemu ... 44
3.6. Testowanie aplikacji ... 46
ZAKOŃCZENIE ... 47
BIBLIOGRAFIA ... 48
AKTY NORMATYWNE ... 49
NETOGRAFIA ... 50
STRESZCZENIE ... 51

WSTĘP

Współczesne usługi cyfrowe w coraz większym stopniu przenoszone są do środowiska internetowego. Dotyczy to również branży rozrywkowej, w tym kin i teatrów, które umożliwiają zakup biletów oraz rezerwację miejsc za pośrednictwem aplikacji webowych. Rozwiązania tego typu muszą być intuicyjne, szybkie oraz niezawodne, ponieważ bezpośrednio wpływają na komfort użytkownika.

Systemy rezerwacyjne stanowią przykład aplikacji, które łączą interfejs użytkownika, logikę biznesową oraz bazę danych w spójny system informatyczny. Ich poprawne zaprojektowanie wymaga uwzględnienia zarówno aspektów technicznych, jak i użytecznościowych. Kluczowe znaczenie ma synchronizacja danych w czasie rzeczywistym oraz zapewnienie spójności informacji o dostępności miejsc.

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji webowej do rezerwacji seansów kinowych z wykorzystaniem biblioteki React w warstwie front-endowej oraz frameworka Spring Boot i bazy danych MySQL w warstwie back-endowej.

Projekt ma charakter praktyczny i stanowi działający prototyp systemu rezerwacyjnego. Aplikacja umożliwia przeglądanie repertuaru filmowego, wybór godziny seansu, dynamiczny wybór miejsc, rejestrację i logowanie użytkownika oraz zapis rezerwacji w bazie danych.
W pracy szczególny nacisk położono na integrację warstwy front-end z backendowym interfejsem REST API oraz na poprawne zaprojektowanie struktury danych.

Motywacja wyboru tematu

Wybór tematu pracy wynika z rosnącego znaczenia aplikacji webowych w codziennym funkcjonowaniu usług komercyjnych. Systemy rezerwacyjne są szeroko wykorzystywane w branży kinowej, hotelarskiej oraz transportowej. Ich poprawne działanie ma bezpośredni wpływ na doświadczenie użytkownika oraz efektywność sprzedaży.

Dodatkową motywacją było praktyczne zastosowanie technologii front-endowych i backendowych w jednym projekcie. Połączenie biblioteki React z frameworkiem Spring Boot umożliwia zrozumienie pełnego procesu tworzenia aplikacji webowej - od interfejsu użytkownika po zapis danych w bazie.

Projekt pozwolił również na przeanalizowanie problemu synchronizacji danych w systemach,

w których wielu użytkowników może wykonywać operacje jednocześnie. Przykładem jest próba rezerwacji tego samego miejsca przez dwie osoby w tym samym czasie.

Przedmiot badań

Przedmiotem badań w niniejszej pracy jest aplikacja webowa do rezerwacji seansów kinowych, w szczególności jej warstwa front-endowa wykonana w technologii React oraz sposób integracji z backendowym interfejsem REST API.

Analizie podlega struktura aplikacji, sposób zarządzania stanem, komunikacja z serwerem oraz mechanizm wyboru i zapisu miejsc w bazie danych.

Cel główny badań

Celem głównym pracy jest zaprojektowanie oraz implementacja działającego systemu rezerwacji seansów kinowych w środowisku webowym z wykorzystaniem technologii React, Spring Boot oraz MySQL.

Cele szczegółowe

C1. Analiza istniejących systemów rezerwacyjnych.

C2. Zaprojektowanie struktury aplikacji webowej.

C3. Implementacja interfejsu użytkownika w React.

C4. Implementacja backendowego API w Spring Boot.

C5. Integracja frontendu z backendem.

C6. Zaprojektowanie bazy danych MySQL.

C7. Przeprowadzenie testów funkcjonalnych systemu.

Główny problem badawczy

W jaki sposób zaprojektować i zaimplementować aplikację webową do rezerwacji seansów kinowych, która będzie intuicyjna dla użytkownika oraz poprawnie zintegrowana z backendowym API?

Problemy szczegółowe

P1. Jakie funkcjonalności powinien posiadać system rezerwacji?

P2. Jak zaprojektować przejrzysty interfejs użytkownika?

P3. Jak zapewnić poprawną komunikację z REST API?

P4. Jak przechowywać dane o rezerwacjach w bazie danych?

P5. Jak przetestować poprawność działania aplikacji?

Metody badawcze

W pracy zastosowano metodę projektowo-implementacyjną.
Najpierw przeprowadzono analizę wymagań oraz zaprojektowano strukturę systemu. Następnie wykonano implementację frontendu oraz backendu. Ostatnim etapem było testowanie funkcjonalności aplikacji[1].

W pracy wykorzystano analizę dokumentacji technicznej użytych technologii oraz testy funkcjonalne systemu w środowisku lokalnym.

Struktura pracy

Praca składa się ze wstępu, trzech rozdziałów oraz zakończenia.
W rozdziale pierwszym omówiono zagadnienia teoretyczne związane z aplikacjami webowymi i wykorzystanymi technologiami.
W rozdziale drugim przedstawiono projekt systemu, w tym strukturę bazy danych oraz diagramy UML.
W rozdziale trzecim opisano implementację aplikacji oraz jej testowanie.

[1] Myers G., The Art of Software Testing, Wiley.

ROZDZIAŁ 1. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ ORAZ PODSTAWY TEORETYCZNE

1.1. Systemy rezerwacyjne – analiza i funkcjonalność

Systemy rezerwacyjne odgrywają istotną rolę w nowoczesnych usługach cyfrowych. W szczególności w branży kinowej umożliwiają one użytkownikowi wygodne zaplanowanie wizyty bez konieczności oczekiwania w kolejce do kasy. Rozwiązania tego typu muszą działać stabilnie, ponieważ przetwarzają dane związane z dostępnością miejsc oraz zapisują informacje o dokonanych rezerwacjach.

W praktyce system rezerwacji seansów kinowych powinien umożliwiać użytkownikowi kilka podstawowych operacji. Pierwszym krokiem jest zapoznanie się z repertuarem filmowym. Użytkownik powinien mieć możliwość sprawdzenia tytułu filmu, jego opisu oraz dostępnych godzin projekcji. Następnie powinien wybrać konkretny seans, czyli określoną datę i godzinę wyświetlania.

Kolejnym bardzo ważnym elementem jest wizualizacja sali kinowej. System powinien w czytelny sposób pokazywać, które miejsca są już zajęte, a które pozostają dostępne. W tym miejscu pojawia się istotny problem techniczny – system musi zapewnić aktualność danych. Jeżeli dwóch użytkowników próbuje zarezerwować to samo miejsce w tym samym czasie, backend musi odpowiednio obsłużyć taką sytuację, aby uniknąć konfliktu danych.

W bardziej rozbudowanych systemach stosuje się mechanizmy tymczasowego blokowania miejsc. Oznacza to, że po wybraniu miejsca jest ono rezerwowane na krótki czas, aby użytkownik mógł dokończyć proces zakupu. W niniejszym projekcie zastosowano uproszczony model, w którym system sprawdza zajętość miejsc przed zapisaniem rezerwacji.

Istotnym elementem każdego systemu rezerwacyjnego jest powiązanie rezerwacji z użytkownikiem. Oznacza to, że system musi przechowywać dane użytkownika oraz przypisywać do niego konkretne rezerwacje. Dzięki temu możliwe jest późniejsze wyświetlenie historii rezerwacji oraz usunięcie wybranej pozycji.

System rezerwacyjny powinien również spełniać określone wymagania niefunkcjonalne. Jednym z nich jest wydajność. Użytkownik oczekuje natychmiastowej reakcji interfejsu, zwłaszcza podczas wyboru miejsc. Drugim istotnym elementem jest czytelność interfejsu. Zbyt skomplikowany układ strony może zniechęcić użytkownika i utrudnić dokonanie rezerwacji.

W praktyce systemy rezerwacyjne różnią się poziomem zaawansowania. Najprostsze rozwiązania umożliwiają jedynie zapis miejsca bez płatności online. Bardziej rozwinięte systemy integrują się z zewnętrznymi systemami płatności, generują bilety elektroniczne oraz wysyłają potwierdzenia na adres e-mail.

Projekt opisany w niniejszej pracy stanowi wersję podstawową systemu rezerwacji. Jego

celem było stworzenie działającego prototypu, który pokazuje pełny proces rezerwacyjny: od wyboru filmu, poprzez wybór miejsca, aż do zapisu rezerwacji w bazie danych.

Zrezygnowano z modułu płatności, aby skupić się na kluczowej funkcjonalności systemu oraz integracji warstwy front-end z backendem.

Podczas analizy istniejących rozwiązań można zauważyć, że większość nowoczesnych systemów rezerwacyjnych działa w oparciu o architekturę klient–serwer oraz REST API. Taki model zapewnia elastyczność oraz możliwość dalszego rozwoju systemu. Właśnie dlatego podobne podejście zostało zastosowane w niniejszym projekcie.

Systemy rezerwacyjne muszą być projektowane w sposób przemyślany, ponieważ przetwarzają dane wrażliwe oraz operują na zasobach ograniczonych, jakimi są miejsca na sali kinowej. Błąd w logice działania może prowadzić do podwójnych rezerwacji lub utraty danych. Z tego względu istotne jest poprawne zaprojektowanie struktury danych oraz mechanizmu zapisu rezerwacji.

Podsumowując, system rezerwacji seansów kinowych jest przykładem aplikacji, która łączy w sobie interfejs użytkownika, logikę biznesową oraz bazę danych. Jego poprawne działanie wymaga odpowiedniej architektury oraz synchronizacji danych pomiędzy warstwami systemu.

1.2. Architektura aplikacji webowych i model klient–serwer[2]

Współczesne aplikacje webowe budowane są najczęściej w oparciu o model klient–serwer. Model ten zakłada wyraźny podział systemu na dwie niezależne części: warstwę prezentacji, czyli frontend, oraz warstwę serwerową, czyli backend. Każda z tych warstw ma inne zadania i odpowiada za inny zakres funkcjonalności.

Frontend jest częścią aplikacji, którą użytkownik widzi w przeglądarce internetowej. Odpowiada on za wyświetlanie treści, reagowanie na działania użytkownika oraz wysyłanie zapytań do serwera. Backend natomiast działa po stronie serwera i zajmuje się przetwarzaniem danych, walidacją informacji, komunikacją z bazą danych oraz zwracaniem odpowiedzi do klienta.

Podział ten jest bardzo istotny z punktu widzenia projektowania systemu. Oddzielenie warstwy prezentacji od logiki biznesowej pozwala uniknąć chaosu w kodzie oraz ułatwia rozwój aplikacji w przyszłości. Dzięki temu frontend może być rozwijany niezależnie od backendu, a zmiany w jednej warstwie nie muszą powodować modyfikacji w drugiej.

W niniejszym projekcie zastosowano klasyczną architekturę trójwarstwową. Pierwszą warstwę stanowi interfejs użytkownika wykonany w technologii React. Drugą warstwę stanowi aplikacja serwerowa w Spring Boot, która udostępnia interfejs REST API. Trzecią warstwą jest relacyjna baza danych MySQL, w której przechowywane są informacje o użytkownikach, filmach oraz rezerwacjach.

Architektura trójwarstwowa pozwala na wyraźne rozdzielenie odpowiedzialności. Frontend zajmuje się jedynie prezentacją danych oraz obsługą interakcji użytkownika. Backend odpowiada za przetwarzanie logiki systemu, sprawdzanie poprawności danych oraz zapis informacji do bazy. Baza danych przechowuje dane w sposób trwały i umożliwia ich późniejsze odczytanie.

W systemach rezerwacyjnych taki podział jest szczególnie istotny. Wybór miejsca na sali kinowej jest operacją, która musi zostać poprawnie obsłużona po stronie serwera. Frontend może jedynie pokazać dostępne miejsca, ale to backend decyduje, czy dane miejsce jest rzeczywiście wolne i czy można je zapisać jako rezerwację. Dzięki temu unika się sytuacji, w której dwa różne urządzenia zapisują tę samą rezerwację w tym samym czasie.

Model klient–serwer umożliwia również skalowalność systemu. W przyszłości backend mógłby zostać uruchomiony na osobnym serwerze lub w środowisku chmurowym, a frontend mógłby być hostowany oddzielnie. Taki podział pozwala dostosować wydajność systemu do liczby użytkowników.

Istotnym elementem architektury aplikacji webowych jest również sposób komunikacji pomiędzy warstwami. W niniejszym projekcie zastosowano komunikację opartą na protokole

[2] Fielding R., Architectural Styles and the Design of Network-based Software Architectures, 2000.

HTTP oraz stylu REST. Oznacza to, że frontend wysyła zapytania HTTP do określonych adresów URL, a backend odpowiada danymi w formacie JSON[3].

Dzięki zastosowaniu REST API możliwe jest łatwe rozszerzenie systemu w przyszłości. Na przykład można stworzyć aplikację mobilną, która będzie korzystać z tego samego API bez konieczności modyfikowania logiki backendu. Wystarczy jedynie przygotować nowy interfejs użytkownika, który będzie wysyłał odpowiednie zapytania.

Architektura aplikacji webowej powinna być projektowana w sposób przemyślany. Nieprawidłowy podział odpowiedzialności może prowadzić do trudności w utrzymaniu kodu oraz zwiększonego ryzyka błędów. W projekcie zwrócono uwagę na czytelny podział plików, osobne kontrolery dla różnych funkcjonalności oraz uporządkowaną strukturę komponentów w React.

Ważnym aspektem architektury jest również bezpieczeństwo. W modelu klient–serwer dane przesyłane są przez sieć, dlatego powinny być odpowiednio zabezpieczone. W wersji projektowej zastosowano podstawową autoryzację użytkownika. W bardziej zaawansowanej wersji systemu należałoby zastosować tokeny autoryzacyjne oraz szyfrowanie haseł.

Kolejnym elementem, który należy uwzględnić przy projektowaniu architektury, jest obsługa błędów. Backend powinien zwracać czytelne komunikaty w przypadku niepoprawnych danych, a frontend powinien odpowiednio je wyświetlać. Dzięki temu użytkownik otrzymuje jasną informację o problemie, a system zachowuje stabilność.

Architektura klient–serwer ma również wpływ na wydajność aplikacji. Przeniesienie przetwarzania danych na serwer zmniejsza obciążenie przeglądarki użytkownika. Jednocześnie backend może zostać zoptymalizowany pod kątem obsługi większej liczby zapytań.

Podsumowując, zastosowanie architektury klient–serwer w projekcie systemu rezerwacji seansów kinowych pozwoliło na zachowanie porządku w strukturze aplikacji, wyraźne oddzielenie odpowiedzialności oraz przygotowanie systemu do ewentualnej dalszej rozbudowy. Taki model jest obecnie standardem w budowie nowoczesnych aplikacji webowych i zapewnia elastyczność oraz stabilność działania systemu.

[3]Dokumentacja JSON, https://www.json.org

1.3. Komunikacja REST i wymiana danych w aplikacjach webowych

Jednym z kluczowych elementów nowoczesnych aplikacji webowych jest sposób komunikacji pomiędzy frontendem a backendem. W przypadku systemów rezerwacyjnych komunikacja ta musi być stabilna, przewidywalna oraz odporna na błędy. W niniejszym projekcie zastosowano styl architektoniczny REST, który jest obecnie standardem w projektowaniu interfejsów sieciowych.

REST, czyli Representational State Transfer, opiera się na wykorzystaniu protokołu HTTP do wymiany danych pomiędzy klientem a serwerem. W praktyce oznacza to, że frontend wysyła zapytania HTTP do określonych adresów URL, a backend przetwarza je i zwraca odpowiedź w ustalonym formacie.

W projekcie wykorzystano podstawowe metody HTTP: GET, POST oraz DELETE. Metoda GET służy do pobierania danych, na przykład listy filmów lub listy zajętych miejsc. Metoda POST wykorzystywana jest do tworzenia nowych rezerwacji lub rejestracji użytkownika. Metoda DELETE umożliwia usunięcie istniejącej rezerwacji.

Istotną cechą REST jest bezstanowość. Oznacza to, że każde zapytanie powinno zawierać wszystkie informacje potrzebne do jego przetworzenia. Serwer nie przechowuje informacji o wcześniejszych zapytaniach użytkownika. Takie podejście upraszcza architekturę systemu oraz ułatwia jego skalowanie.

W niniejszym projekcie komunikacja pomiędzy warstwami odbywa się w formacie JSON. Format ten jest czytelny oraz lekki, co sprawia, że dobrze nadaje się do przesyłania danych przez sieć. Struktura JSON pozwala w prosty sposób przekazywać informacje o filmach, godzinach seansów, użytkownikach oraz wybranych miejscach.

Na przykład podczas tworzenia rezerwacji frontend wysyła do backendu dane zawierające identyfikator filmu, godzinę seansu, adres e-mail użytkownika oraz listę wybranych miejsc. Backend przetwarza te dane, sprawdza poprawność informacji oraz zapisuje je w bazie danych.

Poprawne zaprojektowanie API jest niezwykle istotne. Endpointy powinny być logiczne i czytelne. W projekcie zastosowano podział na kontrolery odpowiadające za konkretne obszary funkcjonalności. Dzięki temu struktura API jest uporządkowana i łatwa do zrozumienia.

W systemach rezerwacyjnych szczególnie ważne jest sprawdzanie aktualności danych. Przed zapisaniem nowej rezerwacji backend powinien zweryfikować, czy wybrane miejsca są nadal dostępne. Nawet jeśli frontend wyświetla miejsce jako wolne, w czasie pomiędzy pobraniem danych a wysłaniem zapytania mogło ono zostać zajęte przez innego użytkownika. Dlatego ostateczna decyzja musi należeć do backendu.

REST API umożliwia również odpowiednią obsługę błędów. W przypadku niepoprawnych danych backend może zwrócić odpowiedni kod statusu HTTP, na przykład 400 w przypadku

błędnego zapytania lub 404 w przypadku braku zasobu. Frontend może następnie wyświetlić użytkownikowi stosowny komunikat.

Istotnym elementem komunikacji jest także bezpieczeństwo. Dane przesyłane pomiędzy frontendem i backendem powinny być walidowane po stronie serwera. W projekcie zastosowano podstawową walidację danych wejściowych, aby uniknąć zapisu niepoprawnych informacji w bazie danych.

Zastosowanie REST API pozwala również na łatwą rozbudowę systemu w przyszłości. Jeżeli w przyszłości powstałaby aplikacja mobilna, mogłaby ona korzystać z tych samych endpointów. Dzięki temu logika biznesowa pozostaje w backendzie, a różne interfejsy użytkownika mogą współpracować z jednym źródłem danych.

Komunikacja REST wpływa również na organizację kodu po stronie frontendu. Każde zapytanie HTTP jest obsługiwane w określonym miejscu aplikacji. W projekcie wykorzystano bibliotekę Axios[4], która upraszcza wysyłanie zapytań oraz odbieranie odpowiedzi z serwera. Dzięki temu kod odpowiedzialny za komunikację jest uporządkowany i czytelny.

W kontekście systemu rezerwacji istotne jest również odpowiednie zarządzanie odpowiedziami asynchronicznymi. Zapytania HTTP wykonywane są asynchronicznie, co oznacza, że frontend musi odpowiednio obsłużyć moment oczekiwania na odpowiedź. W projekcie zastosowano mechanizmy aktualizacji stanu aplikacji po otrzymaniu danych z backendu.

Podsumowując, komunikacja REST stanowi fundament działania systemu rezerwacyjnego. Umożliwia ona wymianę danych pomiędzy frontendem i backendem w sposób uporządkowany, bezpieczny oraz skalowalny. Odpowiednie zaprojektowanie API oraz poprawna obsługa zapytań HTTP mają kluczowe znaczenie dla stabilności oraz poprawności działania całej aplikacji.

[4] Axios Documentation, https://axios-http.com

1.4. Technologie front-end wykorzystane w projekcie – React[5] i zarządzanie stanem

Warstwa front-end w niniejszym projekcie została zrealizowana przy użyciu biblioteki React. Wybór tej technologii był uzasadniony jej popularnością, elastycznością oraz podejściem komponentowym, które umożliwia tworzenie czytelnych i modularnych aplikacji webowych.

React jest biblioteką JavaScript służącą do budowy interfejsów użytkownika. Jego podstawową cechą jest podział aplikacji na komponenty. Komponent można rozumieć jako niezależny fragment interfejsu, który posiada własną logikę oraz strukturę wizualną. Takie podejście pozwala na uporządkowanie kodu oraz ułatwia jego dalszy rozwój.

W projekcie systemu rezerwacji zastosowano kilka głównych komponentów, takich jak lista filmów, widok wyboru miejsc, panel logowania oraz sekcja „Moje rezerwacje". Każdy z tych elementów został zaprojektowany jako osobny komponent, co pozwoliło zachować przejrzystość struktury projektu.

Jednym z kluczowych zagadnień w aplikacjach React jest zarządzanie stanem. Stan aplikacji przechowuje dane, które mogą ulegać zmianie w trakcie działania systemu. W przypadku systemu rezerwacyjnego stan obejmuje między innymi listę filmów, wybrane miejsca, dane zalogowanego użytkownika oraz aktualny widok aplikacji.

Zmiana stanu powoduje automatyczne ponowne renderowanie odpowiednich elementów interfejsu. Dzięki temu użytkownik natychmiast widzi efekty swoich działań, na przykład zaznaczenie wybranego miejsca na sali. React wykorzystuje mechanizm wirtualnego DOM, który pozwala na efektywną aktualizację tylko tych fragmentów strony, które rzeczywiście uległy zmianie.

W projekcie zastosowano funkcjonalne komponenty oraz hooki, takie jak useState[6] i useEffect. Hook useState umożliwia przechowywanie zmiennych w obrębie komponentu, natomiast useEffect pozwala wykonywać określone operacje w momencie załadowania komponentu lub zmiany danych.

Przykładowo, po wejściu na stronę z wyborem miejsc komponent wysyła zapytanie do backendu w celu pobrania listy zajętych miejsc. Po otrzymaniu odpowiedzi dane są zapisywane w stanie, a interfejs automatycznie aktualizuje widok sali.

Istotnym elementem aplikacji jest również obsługa zdarzeń. W systemie rezerwacyjnym użytkownik może kliknąć na konkretne miejsce, co powoduje zmianę jego statusu w stanie aplikacji. Logika wyboru miejsca została zaprojektowana w sposób umożliwiający zaznaczenie tylko wolnych pozycji. Miejsca zajęte są wizualnie oznaczone i nie mogą zostać wybrane.

---

[5] React Documentation, https://react.dev
[6] React Hooks Documentation, https://react.dev/reference/react

React umożliwia również przekazywanie danych pomiędzy komponentami za pomocą właściwości, czyli tak zwanych props. W projekcie wykorzystano ten mechanizm do przekazywania informacji o wybranym filmie czy danych użytkownika pomiędzy różnymi częściami aplikacji.

Do komunikacji z backendem wykorzystano bibliotekę Axios. Biblioteka ta upraszcza wysyłanie zapytań HTTP oraz obsługę odpowiedzi serwera. Każde zapytanie jest wykonywane asynchronicznie, co oznacza, że aplikacja nie blokuje interfejsu podczas oczekiwania na odpowiedź.

Ważnym aspektem aplikacji front-end jest również obsługa błędów. W przypadku niepoprawnej odpowiedzi z backendu aplikacja wyświetla użytkownikowi odpowiedni komunikat. Dzięki temu system jest bardziej przejrzysty i odporny na nieprawidłowe dane.

Podczas projektowania interfejsu zwrócono uwagę na jego prostotę. Układ strony jest przejrzysty, a elementy interaktywne są łatwo dostępne. Celem było stworzenie aplikacji, która nie wymaga dodatkowych instrukcji i pozwala użytkownikowi intuicyjnie przejść przez proces rezerwacji.

Zastosowanie React w projekcie pozwoliło na szybkie tworzenie interaktywnych widoków oraz efektywne zarządzanie stanem aplikacji. Podejście komponentowe ułatwiło organizację kodu oraz jego późniejsze modyfikacje.

Podsumowując, wykorzystanie technologii React w warstwie front-end umożliwiło stworzenie dynamicznej i responsywnej aplikacji webowej. Mechanizmy zarządzania stanem, obsługi zdarzeń oraz komunikacji z backendem zapewniają poprawne działanie systemu rezerwacyjnego i stanowią solidną podstawę do jego dalszego rozwoju.

1.5. Technologie back-end oraz baza danych w projekcie

Warstwa serwerowa systemu rezerwacji została zrealizowana przy użyciu frameworka Spring Boot[7]. Wybór tej technologii był uzasadniony jej popularnością w środowisku Java oraz prostotą tworzenia aplikacji REST. Spring Boot umożliwia szybkie skonfigurowanie projektu oraz automatyczne zarządzanie wieloma elementami systemu, takimi jak konfiguracja serwera czy połączenie z bazą danych.

Backend odpowiada za obsługę logiki biznesowej aplikacji. Oznacza to, że przetwarza zapytania wysyłane z frontendu, weryfikuje dane oraz zapisuje informacje w bazie danych. W systemie rezerwacyjnym backend pełni kluczową rolę, ponieważ to on decyduje o tym, czy dana rezerwacja może zostać zapisana.

Struktura backendu została podzielona na kilka głównych elementów: kontrolery, encje oraz repozytoria. Kontrolery odpowiadają za obsługę zapytań HTTP[8] i są pierwszym punktem kontaktu z frontendem. W projekcie zastosowano oddzielne kontrolery dla różnych obszarów funkcjonalności, takich jak obsługa użytkowników, filmów oraz rezerwacji.

AuthController odpowiada za rejestrację oraz logowanie użytkowników. MovieController obsługuje zapytania związane z listą filmów. ReservationController zajmuje się tworzeniem, pobieraniem oraz usuwaniem rezerwacji. Taki podział zwiększa czytelność kodu i pozwala uniknąć mieszania różnych funkcjonalności w jednym miejscu.

Encje w projekcie reprezentują tabele w bazie danych. Zostały one zdefiniowane jako klasy Java, które odpowiadają strukturze danych przechowywanych w MySQL[9]. W systemie występują między innymi encje User, Movie oraz Reservation. Każda z nich zawiera pola odpowiadające kolumnom w tabeli bazy danych.

Relacje między encjami umożliwiają powiązanie rezerwacji z konkretnym użytkownikiem oraz konkretnym filmem. Dzięki temu możliwe jest przechowywanie informacji o tym, kto dokonał rezerwacji oraz na jaki seans. Taka struktura danych pozwala również na łatwe pobieranie historii rezerwacji danego użytkownika.

Do komunikacji z bazą danych wykorzystano JPA[10] (Java Persistence API). Technologia ta umożliwia mapowanie obiektów Java na tabele w relacyjnej bazie danych. Dzięki temu programista nie musi pisać ręcznie zapytań SQL dla podstawowych operacji, takich jak zapis, odczyt czy usunięcie danych.

---

[7] Spring Boot Documentation, https://spring.io/projects/spring-boot
[8] MDN Web Docs – HTTP, https://developer.mozilla.org/en-US/docs/Web/HTTP
[9] MySQL Documentation, https://dev.mysql.com/doc/
[10] Hibernate ORM Documentation, https://hibernate.org/orm/documentation/

Repozytoria w projekcie pełnią funkcję warstwy dostępu do danych. Odpowiadają za wykonywanie operacji na bazie danych. Wykorzystanie repozytoriów pozwala oddzielić logikę biznesową od operacji związanych z bazą danych. Takie podejście zwiększa przejrzystość kodu i ułatwia jego utrzymanie.

Baza danych MySQL została wybrana jako relacyjne rozwiązanie do przechowywania danych systemu. Relacyjny model danych umożliwia definiowanie zależności między tabelami oraz zapewnia spójność informacji. W przypadku systemu rezerwacyjnego istotne jest, aby każda rezerwacja była jednoznacznie powiązana z użytkownikiem oraz filmem.

W systemach rezerwacyjnych szczególnie ważna jest poprawność zapisu danych. Backend musi sprawdzić, czy wybrane miejsca nie są już zajęte przed zapisaniem rezerwacji. W projekcie logika ta została umieszczona po stronie serwera, co zwiększa bezpieczeństwo i zapobiega błędom wynikającym z manipulacji po stronie klienta.

Istotnym elementem backendu jest również obsługa błędów. W przypadku niepoprawnych danych lub braku zasobu system powinien zwrócić odpowiedni komunikat. Dzięki temu frontend może wyświetlić użytkownikowi informację o problemie, a system zachowuje stabilność.

Backend aplikacji został zaprojektowany w sposób umożliwiający jego dalszą rozbudowę. W przyszłości możliwe byłoby dodanie nowych endpointów, takich jak obsługa płatności online czy panel administratora do zarządzania repertuarem.

Zastosowanie Spring Boot oraz JPA pozwoliło stworzyć uporządkowaną i czytelną strukturę projektu. Kod backendu jest podzielony na logiczne części, a każda warstwa ma jasno określoną odpowiedzialność.

Podsumowując, warstwa back-end oraz baza danych stanowią fundament działania systemu rezerwacji. To właśnie backend odpowiada za poprawność operacji, bezpieczeństwo danych oraz integrację z bazą MySQL. Odpowiednie zaprojektowanie tej części systemu było kluczowe dla zapewnienia stabilności oraz prawidłowego działania całej aplikacji.

ROZDZIAŁ 2. PROJEKT SYSTEMU INFORMATYCZNEGO

2.1. Analiza wymagań i założeń projektowych

Projekt systemu rezerwacji seansów kinowych rozpoczął się od szczegółowej analizy wymagań funkcjonalnych oraz określenia podstawowych założeń technicznych. Etap ten był kluczowy, ponieważ od poprawnego zdefiniowania wymagań zależy późniejsza stabilność oraz logika działania całej aplikacji.

Pierwszym krokiem było określenie, jakie problemy ma rozwiązywać system. Głównym celem aplikacji jest umożliwienie użytkownikowi dokonania rezerwacji miejsca na wybrany seans kinowy bez konieczności kontaktu z kasą biletową. System powinien działać w środowisku webowym i być dostępny z poziomu przeglądarki internetowej.

Podstawowe wymagania funkcjonalne obejmują możliwość przeglądania listy filmów, wyboru konkretnego seansu oraz dynamicznego wyboru miejsc na sali. Użytkownik powinien mieć możliwość zobaczenia aktualnej dostępności miejsc, tak aby nie mógł wybrać pozycji już zajętej. Oznacza to, że system musi synchronizować dane pomiędzy frontendem i backendem w czasie rzeczywistym.

Kolejnym wymaganiem było zapewnienie mechanizmu zapisu rezerwacji w bazie danych. Każda rezerwacja powinna zawierać informacje o użytkowniku, filmie, godzinie seansu oraz wybranych miejscach. Dane te muszą być przechowywane w sposób trwały i umożliwiać ich późniejsze odczytanie.

Istotnym elementem analizy było również uwzględnienie sytuacji wyjątkowych. System powinien poprawnie reagować na błędy, takie jak brak połączenia z serwerem, próba zapisania pustej rezerwacji czy konflikt danych w przypadku równoczesnej próby rezerwacji tego samego miejsca przez dwóch użytkowników.

Podczas określania założeń projektowych zdecydowano się na zastosowanie architektury klient–serwer. Wybór ten wynikał z potrzeby wyraźnego oddzielenia warstwy prezentacji od logiki biznesowej. Frontend miał odpowiadać za interakcję z użytkownikiem, natomiast backend za przetwarzanie danych i komunikację z bazą.

W analizie uwzględniono również wymagania niefunkcjonalne. System powinien działać płynnie i reagować na działania użytkownika bez zauważalnych opóźnień. Interfejs powinien być przejrzysty i intuicyjny, tak aby nawet osoba bez doświadczenia technicznego mogła bez problemu dokonać rezerwacji.

Ważnym założeniem było również umożliwienie dalszej rozbudowy systemu. Struktura aplikacji została zaprojektowana w taki sposób, aby w przyszłości możliwe było dodanie

nowych funkcjonalności, takich jak system płatności online, panel administratora czy bardziej zaawansowana autoryzacja.

Na etapie analizy wymagań podjęto także decyzję o wykorzystaniu konkretnych technologii. React został wybrany jako narzędzie do budowy dynamicznego interfejsu użytkownika. Spring Boot został wybrany jako framework do tworzenia REST API. MySQL został zastosowany jako relacyjna baza danych przechowująca informacje o użytkownikach i rezerwacjach.

Analiza wymagań pozwoliła również określić granice projektu. System nie miał być pełnym komercyjnym rozwiązaniem, lecz działającym prototypem pokazującym integrację frontendu z backendem oraz poprawne przetwarzanie danych.

Podsumowując, etap analizy wymagań stanowił fundament dalszych prac projektowych. Pozwolił on określić zakres funkcjonalności, wybrać odpowiednią architekturę oraz zaplanować strukturę systemu w sposób logiczny i uporządkowany.

Uzasadnienie wyboru technologii

Biblioteka React została wybrana ze względu na podejście komponentowe oraz mechanizm wirtualnego DOM, który umożliwia wydajną aktualizację interfejsu. React jest jedną z najczęściej wykorzystywanych technologii front-endowych, co zapewnia dużą dostępność dokumentacji oraz wsparcia społeczności.

Framework Spring Boot został wybrany ze względu na możliwość szybkiego tworzenia aplikacji REST oraz integracji z bazą danych przy użyciu JPA. Rozwiązanie to umożliwia czytelne oddzielenie warstwy kontrolerów, logiki biznesowej oraz repozytoriów.

Relacyjna baza danych MySQL została zastosowana ze względu na stabilność oraz możliwość definiowania zależności pomiędzy danymi. W systemie rezerwacyjnym spójność informacji jest kluczowa, dlatego relacyjny model danych stanowi odpowiednie rozwiązanie.

Komunikacja pomiędzy frontendem i backendem została zrealizowana w oparciu o styl REST, który jest obecnie standardem w projektowaniu interfejsów sieciowych.

2.2. Projekt bazy danych

W projekcie wykorzystano relacyjną bazę danych MySQL. Do komunikacji z bazą użyto technologii JPA w Spring Boot. Każda klasa encji w backendzie odpowiada jednej tabeli w bazie danych. System składa się z trzech głównych tabel: movies, users oraz reservations.

Struktura tabel

1) Struktura tabeli movies

Tabela movies odpowiada klasie Movie.

Nazwa pola	Typ danych	Opis
id	Long	Klucz główny
title	String	Tytuł filmu
description	String	Opis filmu
imageUrl	String	Adres grafiki filmu

Dodatkowo film posiada listę godzin seansów (showtimes).
Pole to zostało oznaczone adnotacją @ElementCollection, co powoduje utworzenie dodatkowej tabeli pomocniczej przechowującej godziny seansów dla danego filmu.
Jeden film może posiadać wiele godzin seansów.

2) Struktura tabeli users

Tabela users odpowiada klasie User.

Nazwa pola	**Typ danych**	**Opis**
id	Long	Klucz główny
email	String	Adres e-mail użytkownika
password	String	Hasło użytkownika

Tabela ta odpowiada za obsługę procesu rejestracji i logowania.

3) Struktura tabeli reservations

Tabela reservations odpowiada klasie Reservation.

Nazwa pola	Typ danych	Opis
id	Long	Klucz główny
movieId	Long	Identyfikator filmu
time	String	Wybrana godzina seansu
userEmail	String	Adres e-mail użytkownika

Rezerwacja zawiera również listę miejsc (seats).
Pole to zostało oznaczone adnotacją @ElementCollection, co powoduje utworzenie dodatkowej tabeli pomocniczej przechowującej numery wybranych miejsc.
Jedna rezerwacja może zawierać wiele miejsc.

Relacje w systemie

W obecnej wersji systemu relacje są zrealizowane w sposób uproszczony.

- Film w rezerwacji identyfikowany jest przez pole movieId.
- Użytkownik identyfikowany jest przez pole userEmail.

Nie zastosowano klasycznych relacji typu @ManyToOne.
Struktura została uproszczona ze względu na charakter projektu oraz zakres pracy inżynierskiej.

Podsumowanie

Baza danych została zaprojektowana w sposób prosty i zgodny ze strukturą encji backendowych.
Zastosowanie JPA umożliwiło automatyczne mapowanie klas na tabele w bazie danych.
Struktura bazy jest wystarczająca do obsługi procesu rezerwacji seansów kinowych w obecnej wersji aplikacji.

2.3. Projekt architektury systemu

Projekt architektury systemu został opracowany w oparciu o model trójwarstwowy, który zapewnia przejrzysty podział odpowiedzialności pomiędzy poszczególnymi elementami aplikacji. Model ten obejmuje warstwę prezentacji, warstwę logiki biznesowej oraz warstwę danych.

Warstwa prezentacji została zrealizowana w technologii React. Odpowiada ona za wyświetlanie danych, obsługę interakcji użytkownika oraz wysyłanie zapytań do backendu. Frontend nie przechowuje danych trwałych, lecz jedynie aktualny stan aplikacji.

Warstwa logiki biznesowej została zaimplementowana w Spring Boot. To właśnie ta część systemu przetwarza zapytania HTTP, weryfikuje dane oraz podejmuje decyzje dotyczące zapisu informacji w bazie danych. Backend pełni rolę pośrednika pomiędzy frontendem a bazą danych.

Warstwa danych opiera się na relacyjnej bazie MySQL. Dane przechowywane są w tabelach powiązanych ze sobą relacjami. Dzięki temu możliwe jest przypisanie rezerwacji do konkretnego użytkownika oraz filmu.

Projekt architektury zakłada wyraźne oddzielenie odpowiedzialności. Frontend nie zawiera logiki związanej z walidacją rezerwacji. Decyzja o zapisaniu danych należy do backendu. Takie rozwiązanie zwiększa bezpieczeństwo oraz spójność danych.

W backendzie zastosowano podział na kontrolery, repozytoria oraz encje. Kontrolery odpowiadają za obsługę zapytań HTTP. Repozytoria umożliwiają wykonywanie operacji na bazie danych. Encje reprezentują strukturę tabel w bazie.

Architektura systemu uwzględnia również komunikację REST. Każdy zasób, taki jak film czy rezerwacja, posiada określony endpoint. Zapytania są przetwarzane w sposób bezstanowy, co oznacza, że każde żądanie zawiera komplet informacji potrzebnych do jego obsługi.

Istotnym elementem architektury jest również obsługa błędów. Backend zwraca odpowiednie kody statusu HTTP, a frontend interpretuje je i wyświetla użytkownikowi komunikaty. Takie rozwiązanie pozwala na utrzymanie stabilności systemu nawet w przypadku nieprawidłowych danych.

Projekt architektury został przygotowany z myślą o możliwości dalszej rozbudowy. W przyszłości możliwe byłoby wprowadzenie dodatkowych modułów, takich jak system płatności czy panel administracyjny, bez konieczności zmiany podstawowej struktury aplikacji.

Architektura systemu została zaprojektowana w sposób umożliwiający łatwe testowanie. Oddzielenie warstw pozwala na sprawdzanie poprawności działania poszczególnych elementów niezależnie od siebie.

Podsumowując, projekt architektury systemu opiera się na sprawdzonych rozwiązaniach stosowanych w nowoczesnych aplikacjach webowych. Wyraźny podział warstw, zastosowanie REST API oraz wykorzystanie relacyjnej bazy danych zapewniają stabilność, czytelność oraz możliwość dalszego rozwoju systemu rezerwacji.

2.4. Diagramy UML

Na Rysunku 1 przedstawiono diagram przypadków użycia

W celu lepszego przedstawienia działania systemu przygotowano diagramy UML. Diagramy pokazują główne elementy aplikacji oraz proces rezerwacji seansu.

W pracy zastosowano:

- diagram przypadków użycia,
- diagram sekwencji,
- diagram komponentów.

Diagramy odnoszą się bezpośrednio do zaimplementowanego systemu (React + Spring Boot + MySQL).

1) Diagram przypadków użycia

Diagram prezentuje jednego aktora – **Użytkownika**, który korzysta z systemu rezerwacji seansów kinowych.

Użytkownik może wykonać następujące działania:

- rejestracja konta,
- logowanie do systemu,
- przeglądanie dostępnych filmów,
- wybór godziny seansu,
- utworzenie rezerwacji,
- przeglądanie własnych rezerwacji,
- usunięcie istniejącej rezerwacji.

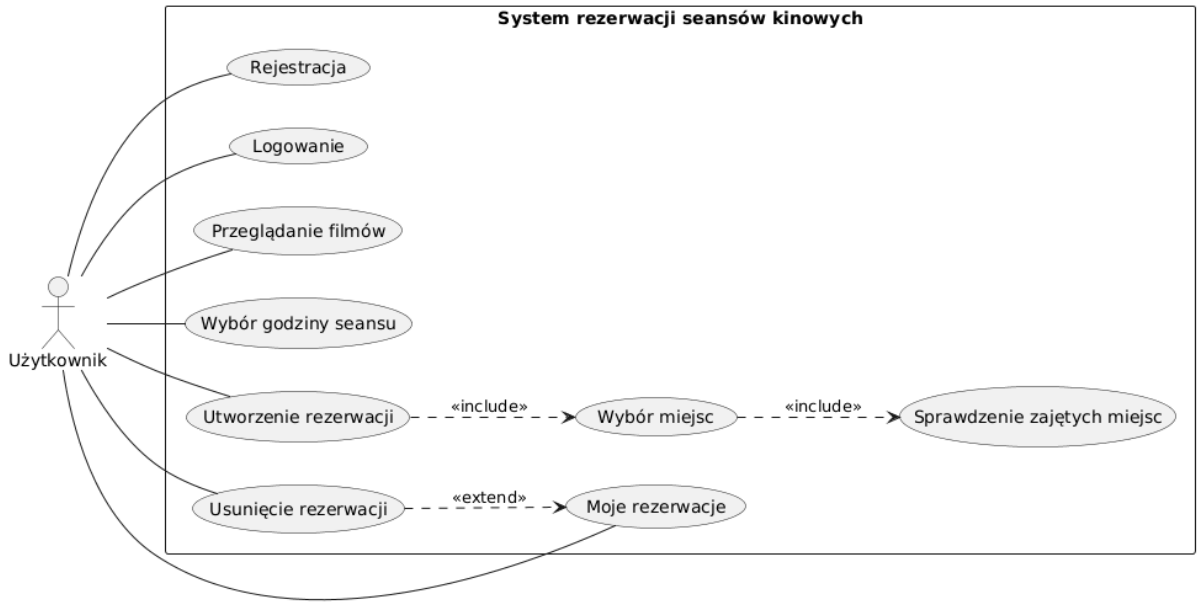
Proces rezerwacji składa się z kilku kroków.

Przypadek użycia **„Utworzenie rezerwacji”** zawiera (<<include>>) przypadek **„Wybór miejsc”**, ponieważ utworzenie rezerwacji wymaga wcześniejszego wyboru miejsc na sali kinowej.

Z kolei przypadek **„Wybór miejsc”** zawiera (<<include>>) przypadek **„Sprawdzenie zajętych miejsc”**, ponieważ przed wyborem miejsc system musi sprawdzić, które miejsca są już zarezerwowane.

Przypadek **„Usunięcie rezerwacji”** rozszerza (<<extend>>) przypadek **„Moje rezerwacje”**, ponieważ użytkownik może usunąć rezerwację podczas przeglądania swoich rezerwacji.

Diagram pokazuje zależności pomiędzy funkcjonalnościami systemu oraz logiczny przebieg procesu rezerwacji.



Rysunek 1. Diagram przypadków użycia systemu rezerwacji seansów kinowych.

2) Diagram sekwencji – proces rezerwacji

Na Rysunku 2 przedstawiono diagram sekwencji pokazujący proces tworzenia rezerwacji w systemie.

W procesie uczestniczą cztery elementy:

- Użytkownik,
- Frontend aplikacji (React),
- Backend aplikacji (Spring Boot),
- Baza danych MySQL.

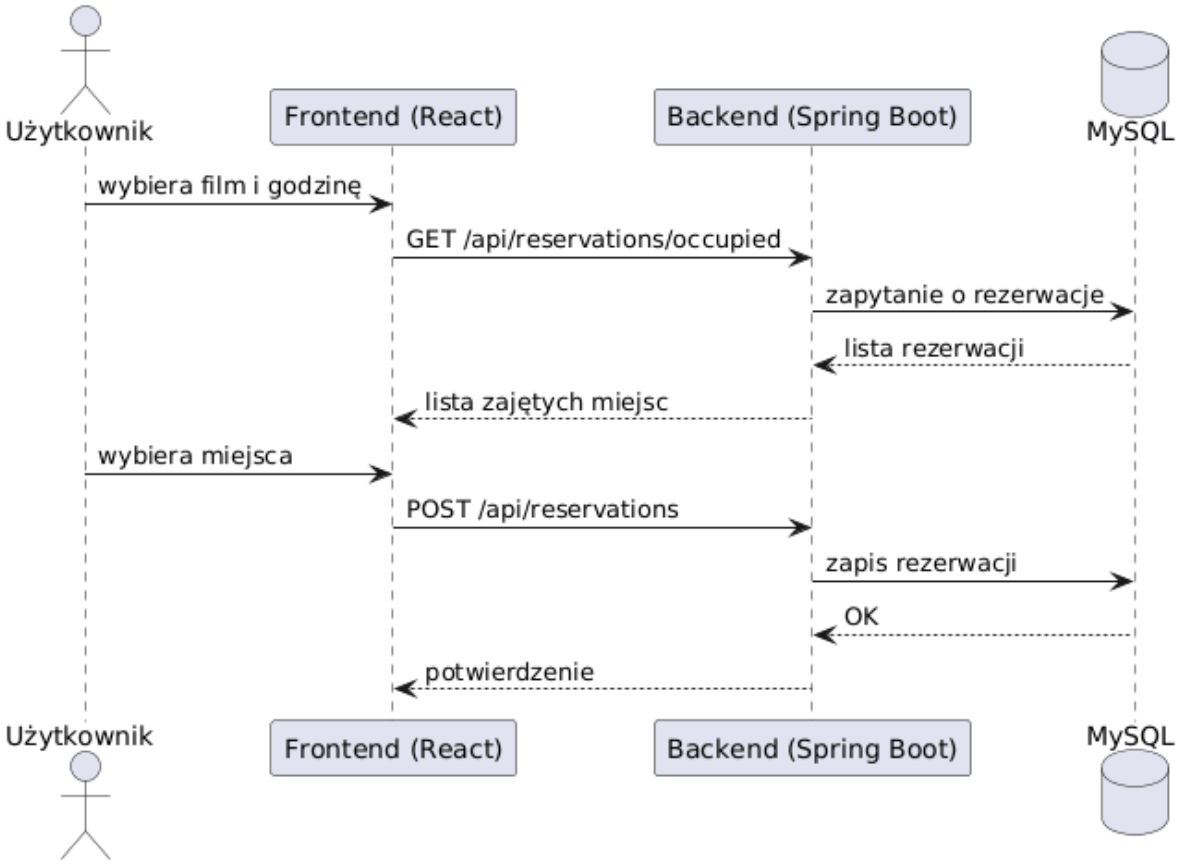
Proces rozpoczyna się od wyboru filmu oraz godziny seansu przez użytkownika. Frontend wysyła zapytanie typu **GET /api/reservations/occupied** do backendu w celu pobrania informacji o zajętych miejscach.

Backend wykonuje zapytanie do bazy danych MySQL i pobiera listę istniejących rezerwacji. Następnie zwraca do frontendu listę zajętych miejsc.

Po wyświetleniu dostępnych miejsc użytkownik dokonuje wyboru miejsc na sali kinowej. Frontend wysyła następnie zapytanie typu **POST /api/reservations**, które zawiera dane rezerwacji.

Backend zapisuje nową rezerwację w bazie danych. Po poprawnym zapisie baza danych zwraca informację potwierdzającą operację (OK). Backend przekazuje potwierdzenie do frontendu, a użytkownik otrzymuje informację o utworzeniu rezerwacji.

Diagram pokazuje pełny przepływ danych pomiędzy warstwą prezentacji, logiki biznesowej oraz bazą danych.



Rysunek 2. Diagram sekwencji procesu rezerwacji seansu kinowego.

3) Diagram komponentów – architektura systemu

Na Rysunku 3 przedstawiono diagram komponentów pokazujący architekturę aplikacji.
System składa się z trzech głównych warstw:

- warstwy prezentacji (Frontend – React),
- warstwy logiki biznesowej (Backend – Spring Boot),
- warstwy danych (MySQL).

W warstwie frontendowej znajdują się główne widoki aplikacji, takie jak:

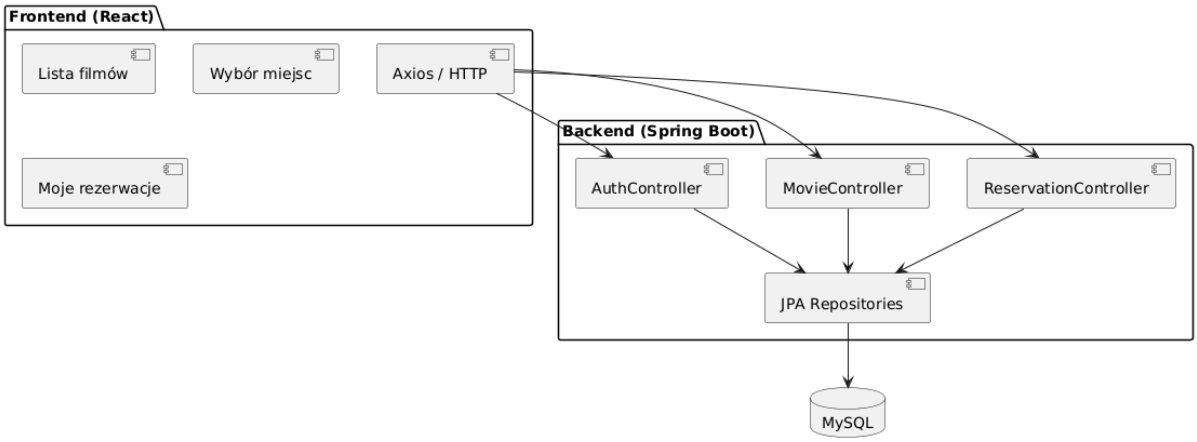
- Lista filmów,
- Wybór miejsc,
- Moje rezerwacje.

Komunikacja z backendem odbywa się przy użyciu biblioteki Axios poprzez zapytania HTTP.
Backend aplikacji został zaimplementowany w technologii Spring Boot.
Zawiera on kontrolery:

- AuthController – odpowiedzialny za rejestrację i logowanie,
- MovieController – obsługujący operacje związane z filmami,
- ReservationController – zarządzający rezerwacjami.

Kontrolery korzystają z warstwy repozytoriów JPA (JPA Repositories), które

umożliwiają komunikację z bazą danych MySQL.
Diagram pokazuje podział odpowiedzialności pomiędzy komponentami systemu oraz sposób komunikacji pomiędzy frontendem, backendem i bazą danych.



Rysunek 3. Diagram komponentów systemu rezerwacji seansów kinowych.

2.5. Projekt interfejsu użytkownika

Projekt interfejsu użytkownika został opracowany z naciskiem na prostotę oraz czytelność. Celem było stworzenie aplikacji, która pozwoli użytkownikowi szybko przejść przez proces rezerwacji bez konieczności zapoznawania się z instrukcją obsługi.

Strona główna aplikacji prezentuje listę dostępnych filmów wraz z podstawowymi informacjami. Każdy film posiada możliwość wyboru konkretnej godziny seansu. Po wybraniu seansu użytkownik przechodzi do widoku sali kinowej.

Widok sali został zaprojektowany jako siatka miejsc. Miejsca dostępne oraz zajęte są oznaczone różnymi kolorami. Taki sposób wizualizacji pozwala użytkownikowi szybko zorientować się w aktualnej dostępności.

Po wybraniu miejsc użytkownik może zatwierdzić rezerwację. System sprawdza poprawność danych oraz zapisuje rezerwację w bazie. Następnie użytkownik może przejść do widoku „Moje rezerwacje”, gdzie wyświetlana jest lista zapisanych pozycji.

Interfejs został zaprojektowany w sposób minimalistyczny. Zrezygnowano z nadmiernej liczby animacji oraz elementów dekoracyjnych. Skupiono się na funkcjonalności oraz przejrzystości.

Podczas projektowania interfejsu uwzględniono również obsługę błędów. W przypadku niepoprawnych danych lub braku połączenia z backendem użytkownik otrzymuje czytelny komunikat.

Projekt interfejsu użytkownika stanowi istotny element systemu, ponieważ to właśnie on wpływa na komfort korzystania z aplikacji. Prosty i logiczny układ elementów zwiększa użyteczność systemu oraz ułatwia jego obsługę.

ROZDZIAŁ 3. IMPLEMENTACJA APLIKACJI

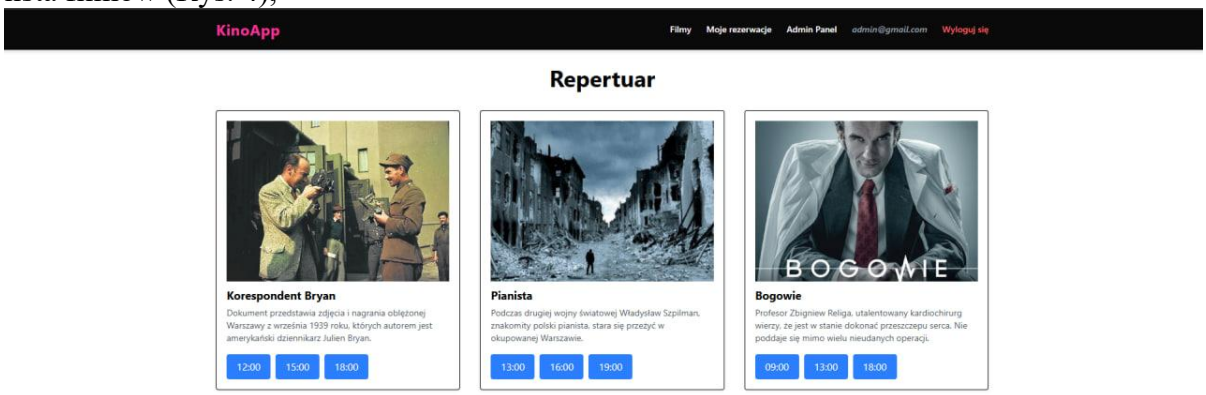
3.1. Struktura aplikacji front-end

Warstwa front-endowa aplikacji została wykonana w bibliotece React. Projekt został zrealizowany jako Single Page Application (SPA). Oznacza to, że po wejściu na stronę aplikacja działa dynamicznie i nie wymaga przeładowania całej strony przy przechodzeniu między widokami. Zmienia się jedynie zawartość interfejsu.

Takie rozwiązanie zostało wybrane, ponieważ aplikacja rezerwacyjna wymaga szybkiej reakcji interfejsu na działania użytkownika. W przypadku wyboru filmu, godziny seansu czy miejsc na sali, użytkownik powinien od razu widzieć efekt swojej akcji. React umożliwia łatwe zarządzanie stanem aplikacji oraz automatyczne odświeżanie widoku po zmianie danych.

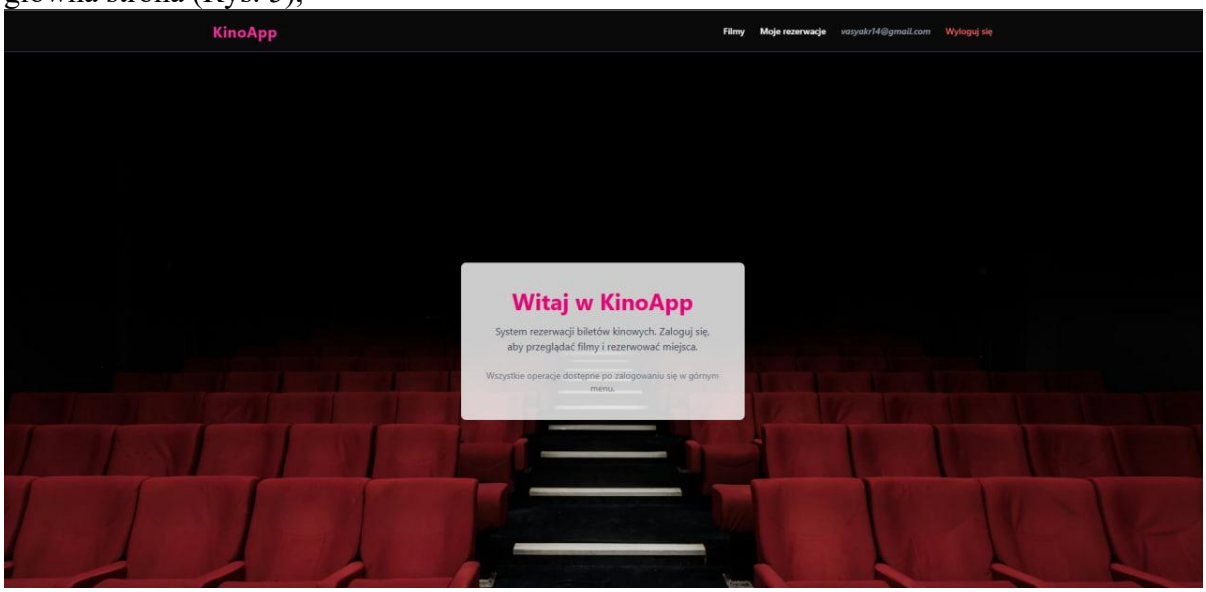
Projekt front-endu został podzielony na kilka głównych widoków:

- lista filmów (Rys. 4),



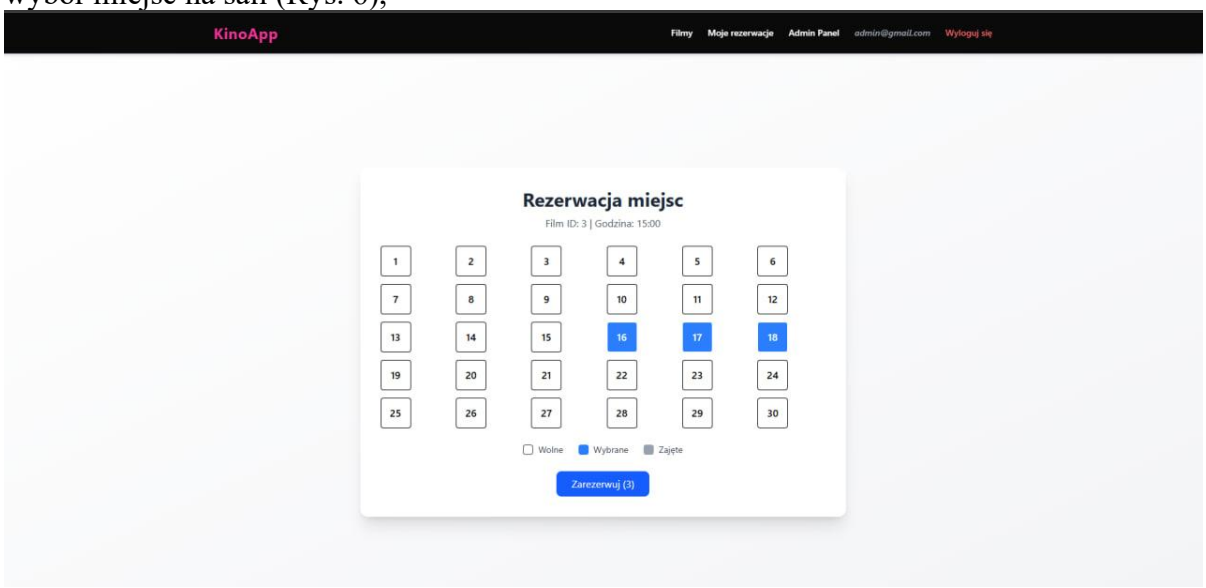
Rysunek 4. Widok listy filmów

- wybór godziny seansu (Rys. 4),
- główna strona (Rys. 5),



Rysunek 5. Widok głównej strony

- wybór miejsc na sali (Rys. 6),

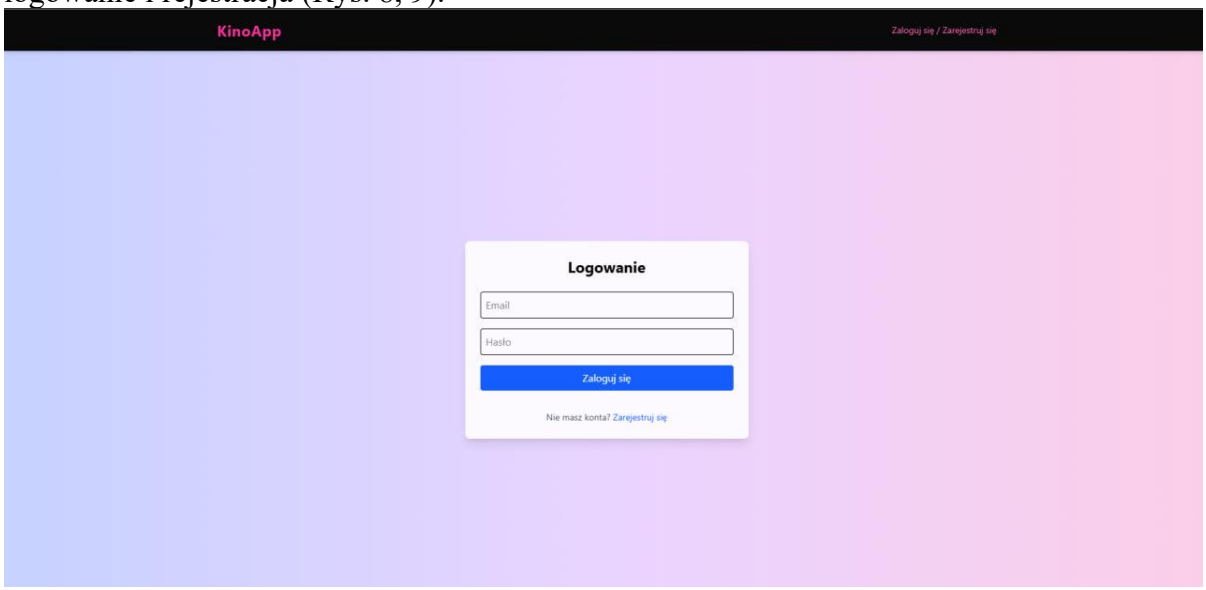


Rysunek 6. Widok wyboru miejsc na sali

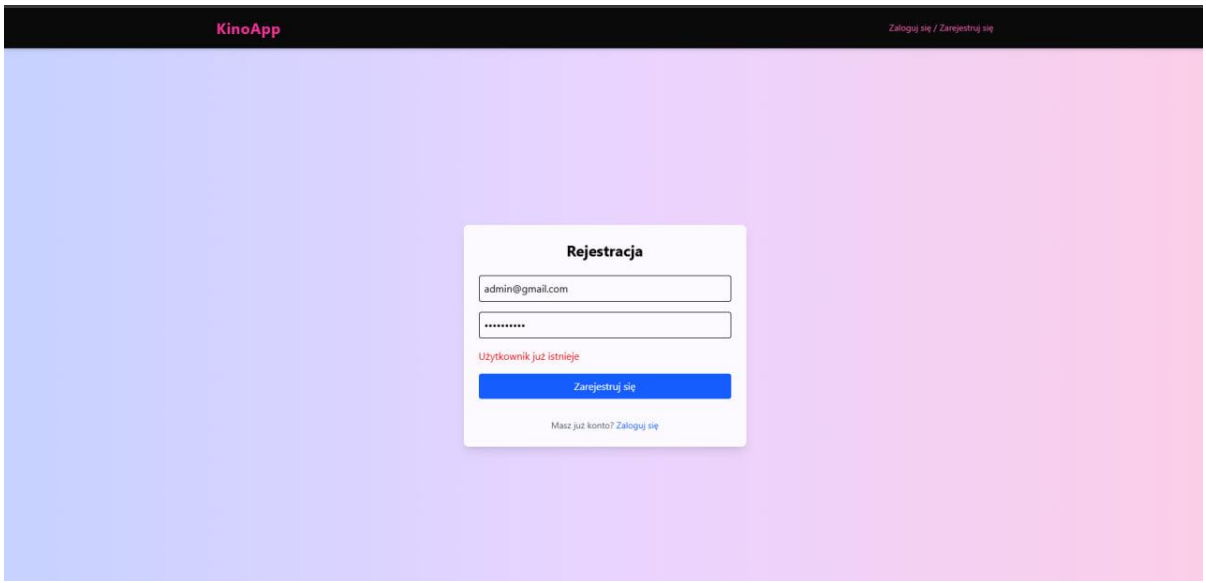
- moje rezerwacje (Rys. 7),

Rysunek 7. Widok moje rezerwacje

- logowanie i rejestracja (Rys. 8, 9).

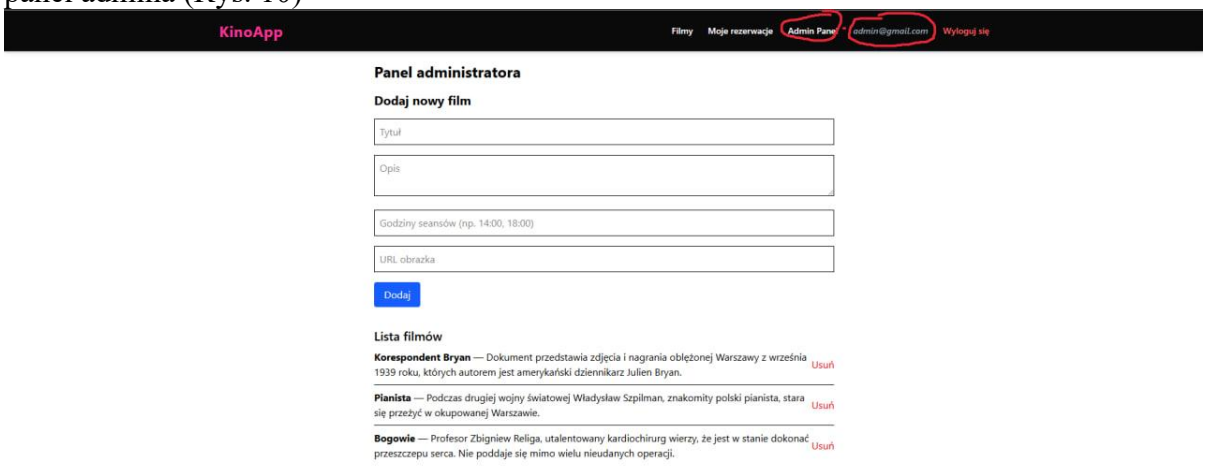


Rysunek 8. Widok logowania



Rysunek 9. Widok rejestracji

- panel admina (Rys. 10)



Rysunek 10. Widok panel admina

Do panelu administratora zawsze ma dostęp konto **admin@gmail.com** - po zalogowaniu automatycznie uzyskuje ono dostęp do dodatkowej podstrony(Rys.10), na której można dodawać i usuwać filmy.

Każdy z tych widoków ma swoją określoną funkcję. Taki podział pozwala zachować porządek w projekcie oraz ułatwia dalsze rozwijanie aplikacji.

W aplikacji zastosowano podejście komponentowe. Oznacza to, że interfejs został podzielony na mniejsze elementy, które można wielokrotnie wykorzystywać. Przykładem

może być:

- komponent karty filmu,
- komponent pojedynczego miejsca na sali,
- komponent formularza logowania,
- komponent listy rezerwacji.

Dzięki takiemu podejściu kod jest bardziej czytelny i modularny. Każdy komponent odpowiada za konkretną część interfejsu i posiada własną logikę.

Zarządzanie stanem aplikacji

W aplikacji wykorzystano mechanizm useState oraz useEffect. Stan aplikacji przechowuje między innymi:

- wybrany film,
- wybraną godzinę,
- listę wybranych miejsc,
- listę zajętych miejsc,
- dane zalogowanego użytkownika.

Zastosowanie stanu pozwala kontrolować zachowanie interfejsu. Na przykład po kliknięciu w miejsce jego kolor zmienia się, ponieważ stan selectedSeats zostaje zaktualizowany.

React automatycznie renderuje ponownie komponent po zmianie stanu. Dzięki temu użytkownik od razu widzi aktualne dane bez ręcznego odświeżania strony.

Routing i przechodzenie między widokami

W aplikacji zastosowano routing po stronie klienta. Oznacza to, że różne adresy URL odpowiadają różnym widokom, jednak wszystko działa w ramach jednej aplikacji.

Przykładowe widoki to:

- strona główna z listą filmów,
- strona wyboru miejsc dla konkretnego filmu,
- strona z rezerwacjami użytkownika.

Routing pozwala zachować logiczną strukturę aplikacji i poprawia czytelność kodu.

Struktura projektu

Projekt front-endowy został podzielony na foldery, które odpowiadają za konkretne elementy aplikacji, na przykład:

- folder z komponentami,
- folder z widokami,
- folder z konfiguracją połączenia z API.

Taki podział sprawia, że projekt jest uporządkowany. W większych aplikacjach brak struktury prowadzi do chaosu w kodzie. W tym projekcie starano się zachować prostotę i czytelność.

Dlaczego React w tym projekcie

React został wybrany[11], ponieważ:

- umożliwia budowę dynamicznego interfejsu,

[11] React Documentation, https://react.dev

- pozwala na łatwe zarządzanie stanem,
- dobrze współpracuje z REST API,
- posiada dużą społeczność i dokumentację.

W kontekście aplikacji rezerwacyjnej React sprawdza się bardzo dobrze, ponieważ umożliwia szybkie reagowanie na wybór użytkownika oraz aktualizację danych w czasie rzeczywistym.

Podsumowanie

Struktura front-endu została zaprojektowana w sposób prosty i logiczny. Aplikacja została podzielona na czytelne widoki i komponenty. Zarządzanie stanem pozwala kontrolować interakcje użytkownika, a architektura SPA zapewnia płynne działanie systemu.

Takie podejście umożliwia dalszy rozwój aplikacji w przyszłości oraz dodawanie nowych funkcji bez konieczności przebudowy całej struktury projektu.

3.2. Logika działania aplikacji – wybór miejsc i interakcja użytkownika

```
const handleSeatClick = (seat) => {
  if (selectedSeats.includes(seat)) {
    setSelectedSeats(selectedSeats.filter(s => s !== seat));
  } else {
    setSelectedSeats([...selectedSeats, seat]);
  }
};
```

Listing 1. Logika wyboru miejsc w React

Najważniejszą funkcją systemu jest możliwość wyboru miejsc na sali kinowej. To właśnie ten element decyduje o tym, czy aplikacja działa poprawnie i czy użytkownik może bez problemu dokonać rezerwacji.

Mechanizm wyboru miejsc został zaimplementowany w oparciu o zmianę stanu komponentu (por. listing 1).

Proces wyboru miejsc składa się z kilku etapów:

1. Użytkownik wybiera film.
2. Użytkownik wybiera godzinę seansu.
3. Aplikacja pobiera z backendu listę zajętych miejsc.
4. Wyświetlana jest siatka miejsc.
5. Użytkownik wybiera wolne miejsca.
6. System zapisuje rezerwację.

Każdy z tych kroków jest kontrolowany przez stan aplikacji.

Generowanie siatki miejsc

W aplikacji sala kinowa została przedstawiona jako siatka numerowanych miejsc. Miejsca są generowane dynamicznie w pętli, co oznacza, że nie są zapisane ręcznie w kodzie.

Przykładowo, jeśli sala posiada 30 miejsc, frontend tworzy listę numerów od 1 do 30 i renderuje je jako elementy interfejsu.

Takie podejście ma kilka zalet:

- łatwo można zmienić liczbę miejsc,
- nie trzeba ręcznie tworzyć każdego elementu,
- kod jest krótszy i bardziej elastyczny.

Stany aplikacji

W logice wyboru miejsc wykorzystano dwa główne stany:

- selectedSeats – lista miejsc wybranych przez użytkownika,

- occupiedSeats – lista miejsc zajętych, pobrana z backendu.

Dzięki rozdzieleniu tych dwóch list można w prosty sposób określić, które miejsce jest:
- wolne,
- wybrane przez użytkownika,
- już zajęte.

Każde miejsce podczas renderowania sprawdza, do której listy należy. Na tej podstawie zmieniany jest jego wygląd (np. kolor).

Obsługa kliknięcia w miejsce

Kliknięcie w miejsce uruchamia funkcję obsługującą zdarzenie. Funkcja ta sprawdza:
1. Czy miejsce znajduje się na liście zajętych – jeśli tak, akcja jest blokowana.
2. Czy miejsce znajduje się już na liście wybranych – jeśli tak, zostaje usunięte.
3. Jeśli miejsce jest wolne – zostaje dodane do listy wybranych.

Synchronizacja z backendem

Po wybraniu filmu i godziny seansu frontend musi pobrać aktualną listę zajętych miejsc. W tym celu wywoływany jest endpoint:
GET /api/reservations/occupied

Zapytanie zawiera parametry:
- movieId
- time

Po otrzymaniu odpowiedzi lista zajętych miejsc zapisywana jest w stanie occupiedSeats.

Dzięki temu aplikacja:
- nie pozwala wybrać zajętego miejsca,
- pokazuje aktualny stan sali,
- synchronizuje dane z bazą.

Aktualizacja interfejsu

React po zmianie stanu automatycznie aktualizuje widok. Oznacza to, że po:
- dodaniu miejsca do selectedSeats,
- usunięciu miejsca,
- pobraniu occupiedSeats,

interfejs natychmiast się odświeża.

Użytkownik od razu widzi:
- które miejsca wybrał,
- które są niedostępne,

- czy jego akcja została zapisana.

Nie ma potrzeby ręcznego odświeżania strony.

Walidacja przed zapisaniem rezerwacji

Przed wysłaniem rezerwacji do backendu wykonywana jest podstawowa walidacja:
- sprawdzenie, czy wybrano godzinę,
- sprawdzenie, czy wybrano przynajmniej jedno miejsce,
- sprawdzenie, czy użytkownik jest zalogowany.

Jeśli któryś z warunków nie jest spełniony, użytkownik otrzymuje komunikat.

Przykładowa walidacja:

```
if (!time) {
  alert("Wybierz godzinę seansu.");
  return;
}

if (selectedSeats.length === 0) {
  alert("Wybierz miejsca.");
  return;
}
```

Takie rozwiązanie zapobiega wysyłaniu niepoprawnych danych do backendu.

Możliwe problemy i ich rozwiązanie

Podczas projektowania logiki wyboru miejsc należało uwzględnić kilka problemów:

1. Dwie osoby mogą próbować wybrać to samo miejsce w tym samym czasie.
   Rozwiązaniem jest ponowne pobranie zajętych miejsc po zapisaniu rezerwacji.

2. Użytkownik może szybko klikać w wiele miejsc.
   React obsługuje to poprawnie dzięki aktualizacji stanu.

3. Brak połączenia z backendem.
   W takim przypadku aplikacja może wyświetlić komunikat o błędzie.

Podsumowanie logiki działania

Logika wyboru miejsc została zaprojektowana w sposób prosty i czytelny. Wykorzystano podstawowe mechanizmy React, takie jak stan komponentu oraz obsługa zdarzeń. System reaguje na działania użytkownika w czasie rzeczywistym, a synchronizacja z backendem zapewnia aktualność danych. Takie rozwiązanie jest wystarczające dla systemu rezerwacji w obecnej wersji aplikacji.

3.3. Komunikacja z backendem i przetwarzanie danych

Aplikacja front-endowa komunikuje się z backendem przy użyciu zapytań HTTP. Backend został zrealizowany w technologii Spring Boot i udostępnia REST API[12]. Dzięki temu frontend może pobierać dane oraz zapisywać rezerwacje w bazie danych.

Komunikacja odbywa się w formacie JSON. Dane są przesyłane między warstwą front-end i back-end w sposób czytelny i uporządkowany.

```
@PostMapping
public Movie addMovie(@RequestBody Movie movie) {
    return movieRepository.save(movie);
}
```

Listing 2. Metoda dodawania filmu w MovieController

```
@GetMapping
public List<Movie> getAllMovies() {
    return movieRepository.findAll();
}
```

Listing 3. Endpoint GET zwracający listę filmów

Struktura REST API

W projekcie wykorzystano kilka podstawowych endpointów.

Operacje związane z filmami:

- GET /api/movies – pobranie listy filmów
- POST /api/movies – dodanie nowego filmu
- DELETE /api/movies/{id} – usunięcie filmu

Operacje związane z rezerwacjami:

- GET /api/reservations/occupied – pobranie zajętych miejsc
- POST /api/reservations – zapis nowej rezerwacji
- GET /api/reservations/user/{email} – pobranie rezerwacji użytkownika
- DELETE /api/reservations/{id} – usunięcie rezerwacji

[12] Fielding R., REST – Representational State Transfer, 2000.

Operacje związane z użytkownikiem:

- POST /api/auth/register – rejestracja
- POST /api/auth/login – logowanie

Podział na osobne kontrolery (MovieController, ReservationController, AuthController) pozwala zachować porządek w kodzie backendu.

Jak pokazano w listingu 2, film jest zapisywany w bazie danych przy użyciu repozytorium JPA.

```
@Entity
public class Movie {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @ElementCollection
    private List<String> showtimes;
}
```

Listing 4. Fragment encji Movie - dane filmu oraz godziny seansów

Przetwarzanie odpowiedzi z backendu

Frontend po otrzymaniu odpowiedzi z backendu zapisuje dane w stanie aplikacji.

Frontend wykorzystuje endpoint pokazany w listingu 3 do wyświetlenia repertuaru.

Przykładowa odpowiedź dla listy filmów:

```
[
  {
    "id": 1,
    "title": "Dune",
    "description": "Opis filmu",
    "showtimes": ["15:00", "18:00"],
    "imageUrl": "..."
  }
]
```

Na podstawie tej odpowiedzi frontend generuje listę kart filmów. Każda karta zawiera tytuł, opis oraz dostępne godziny seansów.

Godziny seansów są przechowywane w polu showtimes (por. listing 4).

W przypadku zajętych miejsc backend zwraca listę numerów miejsc:

[5, 6, 7, 12]

Frontend zapisuje tę listę w stanie occupiedSeats. Następnie podczas renderowania sprawdzane jest, czy dane miejsce znajduje się na tej liście.

Tworzenie rezerwacji

```
@PostMapping
public Reservation createReservation(@RequestBody Reservation reservation) {
    return reservationRepository.save(reservation);
}
```

Listing 5. Tworzenie rezerwacji w ReservationController

Podczas zapisu rezerwacji frontend wysyła zapytanie typu POST z danymi w formacie JSON.

Przykładowe dane wysyłane do backendu:

```
{
  "movieId": 1,
  "time": "18:00",
  "userEmail": "vasyl159856@gmail.com",
  "seats": [5, 6]
}
```

Backend zapisuje dane w bazie MySQL. Encja Reservation przechowuje podstawowe informacje o rezerwacji, natomiast lista miejsc jest zapisywana jako kolekcja.

Po poprawnym zapisie backend zwraca odpowiedź HTTP 200 (OK). Frontend może wtedy wyświetlić komunikat potwierdzający.

Zapis rezerwacji do bazy danych realizowany jest przez metodę przedstawioną w listingu 5.

Obsługa błędów

Podczas komunikacji z backendem mogą wystąpić różne problemy:

- brak połączenia z serwerem,
- błąd walidacji danych,
- próba zapisania niepoprawnej rezerwacji,
- konflikt danych (np. miejsce zostało już zajęte).

W aplikacji zastosowano podstawową obsługę błędów. W przypadku problemu użytkownik otrzymuje komunikat, np.:

- „Nie udało się pobrać danych.”
- „Wystąpił błąd podczas zapisu rezerwacji.”

W przyszłości można rozbudować system o bardziej szczegółowe komunikaty oraz obsługę wyjątków po stronie backendu.

Rola backendu w systemie

Backend odpowiada za:

- przechowywanie danych w bazie MySQL,
- przetwarzanie zapytań HTTP,
- obliczanie zajętych miejsc,
- zapis i usuwanie rezerwacji,
- obsługę rejestracji i logowania.

Frontend nie ma bezpośredniego dostępu do bazy danych. Komunikacja odbywa się wyłącznie przez REST API. Takie rozwiązanie zwiększa bezpieczeństwo oraz porządek w architekturze systemu

Podsumowanie komunikacji

Komunikacja pomiędzy frontendem i backendem została zrealizowana w sposób prosty i zgodny z zasadami REST. Dane są przesyłane w formacie JSON, a backend odpowiada za ich przetwarzanie oraz zapis w bazie danych.

Podział na warstwę prezentacji (React) oraz warstwę logiki biznesowej (Spring Boot) pozwala zachować przejrzystość projektu i ułatwia jego dalszy rozwój.

3.4. Autoryzacja i obsługa użytkownika

W aplikacji zaimplementowano podstawowy mechanizm rejestracji oraz logowania użytkownika. Funkcja ta jest potrzebna, aby rezerwacje były przypisane do konkretnej osoby.

System nie jest rozbudowanym systemem bezpieczeństwa, ale spełnia wymagania projektu inżynierskiego.

Rejestracja użytkownika

Podczas rejestracji użytkownik podaje:
- adres e-mail,
- hasło.

Dane są wysyłane do backendu przez endpoint:
POST /api/auth/register

Backend zapisuje nowego użytkownika w tabeli users.
Encja User zawiera pola:
- id,
- email,
- password.

Po poprawnej rejestracji użytkownik może się zalogować.

Logowanie użytkownika

Logowanie odbywa się przez endpoint:
POST /api/auth/login

Frontend wysyła dane logowania w formacie JSON.
Backend sprawdza, czy użytkownik o podanym adresie e-mail istnieje oraz czy hasło jest poprawne.

Jeśli dane są poprawne, użytkownik otrzymuje odpowiedź pozytywną. W prostszej wersji systemu odpowiedzią może być komunikat potwierdzający.

Po zalogowaniu frontend przechowuje adres e-mail użytkownika w stanie aplikacji. Może on być również zapisany w pamięci przeglądarki (np. localStorage), aby użytkownik nie musiał logować się ponownie po odświeżeniu strony.

Powiązanie rezerwacji z użytkownikiem

Podczas tworzenia rezerwacji do backendu wysyłane jest pole userEmail.
Dzięki temu każda rezerwacja jest powiązana z konkretnym użytkownikiem.
Przykład danych wysyłanych do backendu:

```
{
  "movieId": 1,
  "time": "18:00",
  "userEmail": "vasya@gmail.com",
  "seats": [5, 6]
}
```

W bazie danych rezerwacja przechowuje adres e-mail użytkownika.
Dzięki temu możliwe jest późniejsze pobranie wszystkich rezerwacji danego użytkownika przez endpoint:
GET /api/reservations/user/{email}
Frontend wykorzystuje ten endpoint w widoku „Moje rezerwacje”.

Ograniczenia obecnego rozwiązania

W obecnej wersji aplikacji mechanizm autoryzacji jest uproszczony. Nie zastosowano tokenów JWT ani zaawansowanego systemu sesji.
Hasło użytkownika jest przechowywane w bazie w formie tekstowej (w wersji projektowej).
W rzeczywistym systemie produkcyjnym należałoby zastosować szyfrowanie haseł (np. BCrypt).
Mimo uproszczeń, obecne rozwiązanie jest wystarczające dla projektu inżynierskiego i pozwala na poprawne działanie systemu rezerwacji.

Znaczenie autoryzacji w systemie

Mechanizm logowania pozwala:

- przypisać rezerwacje do użytkownika,
- wyświetlić historię rezerwacji,
- umożliwić usuwanie własnych rezerwacji,
- ograniczyć dostęp do funkcji systemu.

Dzięki temu aplikacja nie jest anonimowa i działa w sposób bardziej realistyczny.

Podsumowanie

Autoryzacja została zaimplementowana w sposób podstawowy, ale funkcjonalny.
Użytkownik może się zarejestrować, zalogować oraz zarządzać swoimi rezerwacjami.
Mechanizm ten współpracuje bezpośrednio z backendem i bazą danych.
Rozwiązanie można w przyszłości rozbudować o bardziej zaawansowane zabezpieczenia, jednak obecna wersja spełnia wymagania projektu.

3.5. Analiza techniczna i możliwości rozwoju systemu

W niniejszym projekcie zrealizowano działającą aplikację do rezerwacji seansów kinowych. System został podzielony na warstwę front-end (React) oraz warstwę back-end (Spring Boot + MySQL). Takie podejście pozwoliło zachować przejrzystą architekturę oraz oddzielić interfejs użytkownika od logiki biznesowej.

Ocena przyjętej architektury

Zastosowany podział na frontend i backend jest rozwiązaniem standardowym w nowoczesnych aplikacjach webowych. Frontend odpowiada za interakcję z użytkownikiem oraz prezentację danych, natomiast backend zajmuje się przetwarzaniem danych i komunikacją z bazą.

Zaletą takiego rozwiązania jest:

- czytelny podział odpowiedzialności,
- możliwość niezależnego rozwoju obu warstw,
- łatwość rozbudowy systemu w przyszłości,
- możliwość podłączenia innego frontendu (np. aplikacji mobilnej).

W projekcie wykorzystano REST API, co pozwala na prostą komunikację między warstwami przy użyciu formatu JSON.

Mocne strony projektu

Najważniejsze mocne strony aplikacji to:

1) Prosta i czytelna struktura kodu.
2) Dynamiczna aktualizacja interfejsu dzięki React.
3) Synchronizacja z backendem przy wyborze miejsc.
4) Możliwość logowania i przypisania rezerwacji do użytkownika.
5) Oddzielenie logiki wyboru miejsc od logiki zapisu danych.

Logika wyboru miejsc została zaprojektowana w sposób przejrzysty. Użytkownik nie może wybrać miejsca już zajętego, a interfejs reaguje natychmiast po kliknięciu.

System spełnia podstawowe wymagania aplikacji rezerwacyjnej.

Ograniczenia obecnej wersji

Projekt ma również pewne ograniczenia, które wynikają z jego charakteru edukacyjnego.

1. Brak zaawansowanego systemu bezpieczeństwa (np. JWT).
2. Hasła nie są szyfrowane (w wersji projektowej).
3. Brak systemu płatności online.

4. Brak podziału sali na rzędy i sektory (miejsca są numerowane liniowo).
5. Brak obsługi administratora do zarządzania repertuarem w pełnym zakresie.

W praktycznym zastosowaniu należałoby rozbudować system o dodatkowe zabezpieczenia oraz walidację po stronie serwera.

Możliwości rozwoju aplikacji

W przyszłości system może zostać rozbudowany o:

- system płatności online (np. integracja z zewnętrznym API),
- zastosowanie tokenów JWT do bezpiecznej autoryzacji,
- szyfrowanie haseł użytkowników,
- panel administratora do zarządzania filmami i seansami,
- podział sali na rzędy i kolumny,
- wysyłanie potwierdzeń rezerwacji na e-mail,
- aplikację mobilną wykorzystującą to samo API.

Dzięki zastosowaniu architektury opartej na REST API możliwe jest dalsze rozwijanie systemu bez konieczności przebudowy całej aplikacji.

Wnioski techniczne

Projekt pokazał, że React jest odpowiednim narzędziem do budowy dynamicznych aplikacji webowych. Umożliwia szybkie reagowanie na działania użytkownika oraz czytelne zarządzanie stanem.
Spring Boot w połączeniu z JPA pozwala w prosty sposób tworzyć API oraz zarządzać danymi w bazie MySQL. Dzięki temu backend może być napisany w sposób uporządkowany i czytelny.
Całość systemu działa poprawnie w środowisku lokalnym i spełnia założenia projektu inżynierskiego.

Podsumowanie rozdziału

W rozdziale trzecim przedstawiono implementację aplikacji, strukturę front-endu, logikę działania, komunikację z backendem oraz mechanizm autoryzacji. Omówiono również mocne strony systemu oraz możliwe kierunki jego rozwoju.

Zrealizowany projekt stanowi kompletny system rezerwacji seansów kinowych w wersji podstawowej i może być rozwijany w przyszłości o kolejne funkcjonalności.

3.6. Testowanie aplikacji

Testowanie aplikacji przeprowadzono w środowisku lokalnym. Weryfikacji poddano najważniejsze funkcjonalności systemu.

Pierwszym etapem było sprawdzenie poprawności procesu rejestracji i logowania użytkownika. Przetestowano możliwość utworzenia nowego konta oraz ponownego zalogowania przy użyciu zapisanych danych.

Następnie przeprowadzono test procesu rezerwacji. Sprawdzono, czy po wyborze filmu oraz godziny seansu system poprawnie pobiera listę zajętych miejsc z backendu. Zweryfikowano również, czy wybrane miejsca są zapisywane w bazie danych.

Przeprowadzono test konfliktu danych poprzez próbę rezerwacji tego samego miejsca w dwóch sesjach przeglądarki. Backend poprawnie przetwarza dane i zapisuje rezerwację w sposób spójny.

Sprawdzono także funkcję usuwania rezerwacji oraz poprawność wyświetlania listy „Moje rezerwacje”.

Testy wykazały poprawne działanie aplikacji w zakresie założonych funkcjonalności.

ZAKOŃCZENIE

Celem niniejszej pracy było zaprojektowanie oraz implementacja aplikacji webowej do rezerwacji seansów kinowych z wykorzystaniem technologii React, Spring Boot oraz MySQL. Założony cel został osiągnięty poprzez stworzenie działającego systemu umożliwiającego przeglądanie repertuaru, wybór miejsc oraz zapis rezerwacji w bazie danych.

W pracy przedstawiono zarówno część teoretyczną, jak i projektową. W pierwszym rozdziale omówiono podstawy architektury aplikacji webowych oraz technologie wykorzystane w projekcie. W drugim rozdziale zaprezentowano projekt systemu, strukturę bazy danych oraz diagramy UML. Trzeci rozdział poświęcono implementacji aplikacji oraz analizie działania poszczególnych elementów systemu.

Szczególną uwagę poświęcono warstwie front-end, w której zaimplementowano dynamiczny mechanizm wyboru miejsc oraz zarządzanie stanem aplikacji. Komunikacja z backendem została zrealizowana przy użyciu REST API i formatu JSON. Backend odpowiada za walidację danych oraz zapis informacji w relacyjnej bazie danych.

Zrealizowany projekt stanowi funkcjonalny prototyp systemu rezerwacyjnego. Umożliwia on przeprowadzenie pełnego procesu rezerwacji bez modułu płatności online. Architektura aplikacji została zaprojektowana w sposób umożliwiający dalszą rozbudowę systemu.

W przyszłości możliwe jest rozszerzenie projektu o dodatkowe funkcjonalności, takie jak integracja z systemem płatności, wprowadzenie tokenów autoryzacyjnych JWT, szyfrowanie haseł użytkowników czy panel administracyjny.

Praca pozwoliła na praktyczne zastosowanie wiedzy z zakresu tworzenia aplikacji webowych, projektowania architektury klient–serwer oraz integracji frontendu z backendem. Zastosowane rozwiązania spełniają założone wymagania funkcjonalne i stanowią podstawę do dalszego rozwoju systemu.

BIBLIOGRAFIA

Flanagan D., JavaScript. The Definitive Guide, O'Reilly Media, 2020.

Banks A., Porcello E., Learning React. Modern Patterns for Developing React Apps, O'Reilly Media, 2020.

Freeman E., Robson E., Head First Design Patterns, O'Reilly Media, 2021.

Sommerville I., Software Engineering, Pearson Education, 2016.

Pressman R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill, 2019.

Myers G., The Art of Software Testing, Wiley.

Fielding R., REST – Representational State Transfer, 2000.

Fielding R., Architectural Styles and the Design of Network-based Software Architectures, 2000.

AKTY NORMATYWNE

1. **Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych i w sprawie swobodnego przepływu takich danych (RODO).**

2. **Ustawa z dnia 18 lipca 2002 r. o świadczeniu usług drogą elektroniczną (Dz.U. 2002 nr 144 poz. 1204 z późn. zm.).**

3. **Ustawa z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz.U. 1994 nr 24 poz. 83 z późn. zm.).**

4. **Ustawa z dnia 10 maja 2018 r. o ochronie danych osobowych (Dz.U. 2018 poz. 1000).**

NETOGRAFIA

React – Oficjalna dokumentacja, https://react.dev

Spring Boot – Oficjalna dokumentacja, https://spring.io/projects/spring-boot

JSON – W3C Specification, https://www.json.org

Hibernate ORM Documentation, https://hibernate.org/orm/

JWT Introduction, https://jwt.io/introduction

Axios docs, https://axios-http.com/

React Hooks Documentation, https://react.dev/reference/react

MySQL Documentation, https://dev.mysql.com/doc/

MDN Web Docs – HTTP, https://developer.mozilla.org/en-US/docs/Web/HTTP

STRESZCZENIE

Praca przedstawia projekt oraz implementację aplikacji webowej do rezerwacji seansów kinowych. System został wykonany w architekturze klient–serwer z wykorzystaniem technologii React po stronie front-end oraz Spring Boot i MySQL po stronie back-end.

Aplikacja umożliwia przeglądanie repertuaru filmowego, wybór godziny seansu, dynamiczny wybór miejsc oraz zapis rezerwacji w bazie danych. Zaimplementowano również podstawowy mechanizm rejestracji i logowania użytkownika.

W pracy omówiono analizę wymagań systemu, projekt bazy danych, diagramy UML oraz szczegóły implementacyjne. Szczególną uwagę poświęcono mechanizmowi synchronizacji danych między frontendem i backendem oraz zarządzaniu stanem aplikacji w React.

Zrealizowany projekt stanowi funkcjonalny prototyp systemu rezerwacyjnego i może zostać w przyszłości rozbudowany o dodatkowe funkcjonalności.